FAST NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES

(FAST – NUCES), LAHORE CAMPUS

Department of Cyber Security

LAB 03

Python Programming for Cyber Security:

From C++ Foundations to Building Offensive Security Tools

Prepared by

Muhammad Annus Shahzad

Roll Number: 23L-3004

Supervised By

Prof. Dr. Arshad Ali, HoS (FSC), NUCES, Lahore

Program: BS Cyber Security (3rd Semester)

Lab Information Sheet

Field	Details
Course Title	Cyber Security
Lab Number	Lab 03
Lab Title	Python Programming for Cyber Security — From C++ Foundations to Building Offensive Security Tools
Semester	3rd Semester, BS Cyber Security
Recommended Duration	3 Hours (may be split across two lab sessions)
Lab Environment	Kali Linux (VirtualBox/VMware VM) — Python 3.x, pip, venv, a text editor (VS Code / nano / vim), terminal
Prerequisites	Lab 01 (Kali Linux installation & VM setup), Lab 02 (Linux shell fundamentals), C++ Programming Fundamentals & Object-Oriented Programming (prior coursework)
Modules / Tools Required	python3, pip3, venv, socket, argparse, concurrent.futures, hashlib, re, requests, nmap (system package)
Prepared By	Muhammad Annus Shahzad (23L-3004) — Creator
Supervised By	Prof. Dr. Arshad Ali

Table of Contents

Lab Information Sheet 2
Table of Contents 3
1. Introduction 5
2. Learning Objectives 5
3. Required Resources & Environment Setup 6
    3.1 Working Inside a Virtual Environment (venv) 7
    3.2 Recommended Editor 7
Part A — From C++ to Python: The Mental Model Shift 8
    A.1 Compiled vs. Interpreted Execution 8
    A.2 Syntax-to-Syntax Translation Table 9
    A.3 Why This Matters for Security Work 9
Part B — Python Fundamentals I: Syntax, Variables, and Data Types 10
    B.1 Running Python 10
    B.2 Variables and Dynamic Typing 10
    B.3 Core Data Types 11
    B.4 Collections: list, tuple, dict, set 11
    B.5 String Formatting 12
    B.6 Input and Output 12
Part C — Control Flow, Functions, and Exception Handling 13
    C.1 Conditionals 13
    C.2 Loops 13
    C.3 Functions 14
    C.4 Lambda Functions 14
    C.5 Exception Handling 14
Part D — Object-Oriented Programming in Python 17
    D.1 Classes, Objects, and Constructors 17
    D.2 Encapsulation 17
    D.3 Inheritance and Polymorphism 18
    D.4 Useful Dunder ('Magic') Methods 18
Part E — Modules, Packages, and the Python Security Ecosystem 19
    E.1 The Import System 19
    E.2 pip and Virtual Environments (Recap) 19
    E.3 Standard Library Highlights for Security Work 19
    E.4 Third-Party Security-Relevant Packages 20
Part F — Networking with Python: Socket Programming Fundamentals 22
    F.1 What Is a Socket? 22
    F.2 The TCP Three-Way Handshake 22
    F.3 The socket Module API 22
    F.4 Writing a Simple TCP Client 23
    F.5 Writing a Simple TCP Echo Server 24
Part G — Introducing Nmap: The Industry-Standard Port Scanner 26
    G.1 What Is Nmap, and Why Learn It First? 26
    G.2 Legal & Ethical Reminder 26
    G.3 Core Nmap Syntax 26
    G.4 Hands-On: Your First Authorized Scan 27
    G.5 The Bridge to Part H 27
Part H — Building Your Own Tool: A Multithreaded Python Port Scanner 28

H.1 Design Overview 28
H.2 Step 1 — A Single-Port Checker Function 28
H.3 Step 2 — A Command-Line Interface with argparse 29
H.4 Step 3 — Multithreading with concurrent.futures 29
H.5 Full Program 30
H.6 Sample Run 31
H.7 Suggested Enhancements (Try These) 32
Part I — Banner Grabbing and the Title-Grabber Tool 33
I.1 What Is Banner Grabbing, and Why Does It Matter? 33
I.2 A Generic TCP Banner Grabber 33
I.3 An HTTP Title Grabber 33
Part J — Other Cyber Security Use Cases for Python 35
J.1 File Integrity Verification with hashlib 35
J.2 Automating External Tools with subprocess 35
J.3 Log Parsing and Analysis with re 35
J.4 Basic Web Reconnaissance with requests 36
J.5 Packet Inspection with Scapy (Conceptual Overview) 36
J.6 Symmetric Encryption with cryptography 37
J.7 A Note on Higher-Risk Tool Categories 37
Part K — Hands-On Graded Exercises 38
Exercise 1: C++-to-Python Warm-Up: Translating a Cipher 38
Exercise 2: Extend the Port Scanner: Banner Grabbing + CSV Output 38
Exercise 3: Bulk HTTP Title Grabber 39
Exercise 4: SSH Auth Log Analyzer 39
Exercise 5: File Integrity Checker 40
Self-Evaluation 41
Skills Checklist 41
Conceptual Questions 41
Practical Self-Test Tasks 42
Self-Grading Guide 42
Deliverables & Rubric 43
References 44

1. Introduction

In Lab 01, you installed and configured Kali Linux inside a virtual environment. In Lab 02, you learned to move through that environment confidently — the shell, the filesystem, permissions, processes, and the everyday command-line toolkit that every security practitioner relies on. Those two labs gave you the operating environment. This lab gives you the programming layer that turns that environment into a platform for building your own tools.

Python is, by a wide margin, the most common language in the cyber security industry. Metasploit modules, Impacket, Scapy, most CTF solve-scripts, the majority of proof-of-concept exploits published on security research blogs, automated OSINT frameworks, and countless internal red-team and blue-team utilities are written in Python. The reason is not that Python is the fastest or the most elegant language — it is that Python removes friction between an idea and a working tool. A security professional rarely needs a program that runs in microseconds; they need a program that exists in the next ten minutes, that talks to a network socket, parses a log file, or automates a repetitive task correctly on the first try.

You are not starting from zero. You have already spent a full semester learning C++ programming fundamentals and Object-Oriented Programming. That is a significant advantage: you already understand variables, control flow, functions, classes, and how a program executes — concepts that do not change from language to language. What changes is syntax, tooling, and idiom. This lab is deliberately structured to keep translating new Python concepts back to the C++ and OOP vocabulary you already own, so that you are learning a new dialect of programming rather than programming itself for the second time.

The lab builds in one continuous direction: general Python syntax → functions, exceptions and OOP → the standard library and security-relevant third-party packages → socket programming and the TCP handshake → Nmap as the industry-standard reference tool → your own hand-built multithreaded port scanner → banner and HTTP title grabbing → a wider tour of Python's other roles in cyber security (hashing, automation, log analysis, packet inspection, and encryption). By the end, you will not just know Python syntax — you will have built, from first principles, some of the same categories of tool that Nmap itself is built from.

Legal & Ethical Note

Every scanning, connection, and banner-grabbing exercise in this lab must be performed only against systems you own, your own lab VMs, your local machine (127.0.0.1), a private lab network, or explicitly authorized public test targets such as scanme.nmap.org (which the Nmap project maintains specifically for this purpose). Scanning or connecting to systems without authorization is illegal in most jurisdictions, including Pakistan under the Prevention of Electronic Crimes Act (PECA) 2016, and violates FAST-NUCES's acceptable use policy. Treat every technique in this manual as something you are learning to use defensively and only with permission.

2. Learning Objectives

By the end of this lab, students will be able to:

- Translate core programming concepts (variables, control flow, functions, exception handling, and classes) from C++ syntax to idiomatic Python syntax.
- Explain the practical differences between compiled and interpreted execution, and why interpreted, dynamically-typed languages are favored for rapid security tool development.

- Use Python's standard library (os, sys, subprocess, re, hashlib, json, argparse, concurrent.futures) to automate common security-relevant tasks.
- Identify and describe the role of key third-party Python packages used across the cyber security industry (scapy, requests, paramiko, cryptography, python-nmap, BeautifulSoup).
- Explain the TCP three-way handshake and implement TCP client and server programs using Python's socket module.
- Operate Nmap to perform authorized host discovery and port/service scanning, and interpret its output.
- Design, implement, and test a multithreaded TCP port scanner in Python from first principles, without relying on any external scanning library.
- Implement a banner-grabbing utility and an HTTP title-grabbing utility to fingerprint services running on open ports.
- Apply Python to at least three additional cyber security use cases: file-integrity hashing, log analysis with regular expressions, and basic symmetric encryption.
- Critically evaluate the ethical and legal boundaries of writing and running network-scanning and reconnaissance tools.

3. Required Resources & Environment Setup

This lab uses the same Kali Linux VM you configured in Lab 01. Python 3 ships with Kali by default. Before continuing, verify your environment from a terminal:

```
# Check Python 3 is installed and note the version
python3 --version

# Check pip (Python's package manager) is available
pip3 --version

# Check that nmap is installed (it ships with Kali by default)
nmap --version
```

Verifying the required tools before starting the lab.

3.1 Working Inside a Virtual Environment (venv)

A Python virtual environment is an isolated space for a project's dependencies — conceptually similar to keeping a separate, self-contained build folder per C++ project instead of installing every library system-wide. It prevents package versions required by one project from clashing with another.

```
# Create a project folder and a virtual environment inside it
mkdir ~/lab03-python && cd ~/lab03-python
python3 -m venv labenv

# Activate the virtual environment
```

```
source labenv/bin/activate
# Your prompt should now begin with (labenv)

# Install the two third-party packages this lab uses
pip install requests cryptography

# Deactivate when finished
deactivate
```

Creating and using a virtual environment for this lab's dependencies.

3.2 Recommended Editor

Any editor is acceptable — VS Code, nano, or vim, all covered conceptually in Lab 02. This manual assumes you can create a .py file, edit it, save it, and run it with python3 filename.py, exactly as you compiled and ran .cpp files with g++ in your programming courses.

Part A — From C++ to Python: The Mental Model Shift

Before writing a single line of Python, it helps to understand what actually changes when you move from C++ to Python, and — just as importantly — what does not. The concepts you learned in your programming and OOP courses (variables hold values, functions take input and return output, classes bundle data with behavior, control flow decides what executes) are language-independent. What changes is how those ideas are expressed, and how the program gets from source code to a running process.

A.1 Compiled vs. Interpreted Execution

C++ is a compiled language: the compiler (g++) translates your entire .cpp file into a native machine-code executable before it ever runs. Any syntax error is caught at compile time, and the resulting binary runs directly on the CPU. Python is an interpreted language: the Python interpreter reads your .py file, translates it on the fly into an intermediate bytecode, and executes that bytecode inside the Python Virtual Machine (PVM). There is no separate compile step from the developer's point of view — you simply run python3 script.py and it executes immediately.

This difference is exactly why Python dominates security tooling. A penetration tester who needs a quick script to test a hypothesis about a target does not want to manage a build system — they want to write eight lines, run them, see the result, and iterate. That tight write-run-observe loop is what makes Python the language of choice for exploit development, automation, and rapid prototyping, even though a compiled language will usually run a given algorithm faster.

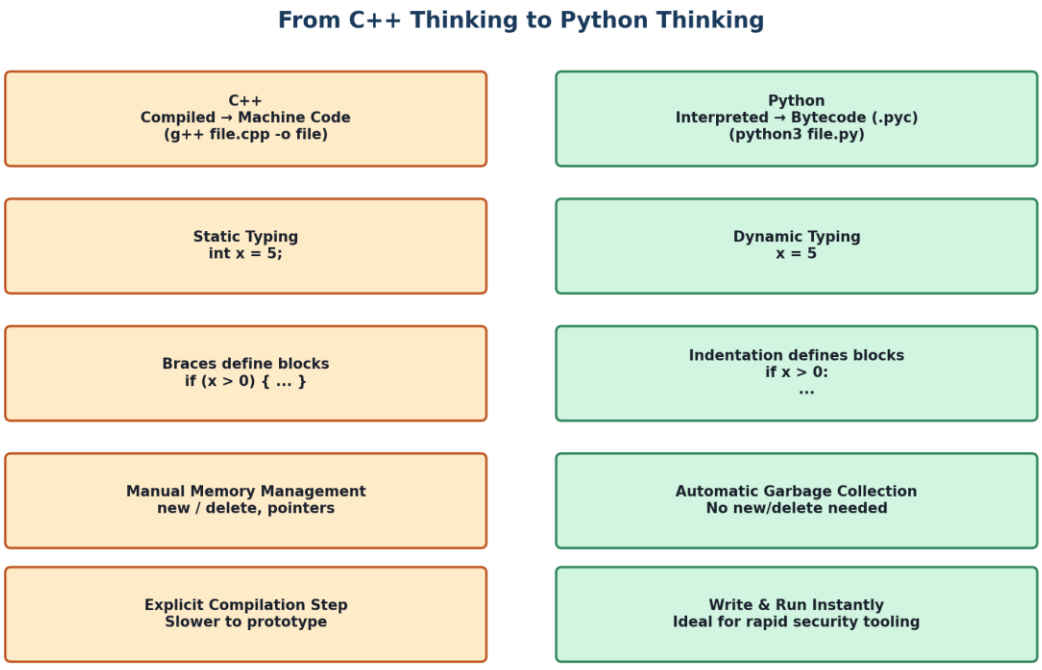


Figure 1: The mental shift from C++'s compiled, statically-typed, brace-delimited style to Python's interpreted, dynamically-typed, indentation-delimited style.

A.2 Syntax-to-Syntax Translation Table

Keep this table close for the rest of the lab. Every new Python construct introduced from here on maps back to something you already wrote in C++.

Concept	C++	Python
Program entry point	`int main() { ... return 0; }`	if __name__ == "__main__": ...
Variable declaration	int x = 5;	x = 5
Printing output	std::cout << x << std::endl;	print(x)
Single-line comment	// comment	# comment
Block delimiter	{ ... }	Indentation (typically 4 spaces)
Statement terminator	; (semicolon required)	Newline (semicolon optional/unused)
Boolean literals	true / false	True / False
Null / empty value	`nullptr`	None
String concatenation	s1 + s2 (std::string)	s1 + s2 or f"{s1}{s2}"
Dynamic array	std::vector<int> v;	v = []
Key-value map	std::map<std::string,int> m;	m = {}
Compiling / running	g++ file.cpp -o file && ./file	python3 file.py
Function definition	int add(int a, int b) { return a+b; }	def add(a, b): return a + b

A.3 Why This Matters for Security Work

Speed of iteration is not a cosmetic convenience in security — it is often the deciding factor in whether a defender catches an incident in time, or whether a researcher can validate a vulnerability before a disclosure window closes. Across the rest of this lab, every tool you build (a port scanner, a banner grabber, a log analyzer) is small enough to write, test, and fix within a single lab session precisely because Python removes the compile-link-run cycle from the equation.

Part B — Python Fundamentals I: Syntax, Variables, and Data Types

B.1 Running Python

There are two ways to run Python, both of which you will use in this lab:

- Interactive mode (the REPL): type python3 in a terminal and execute statements one at a time — useful for quick experiments, similar to testing a small expression before committing it to a full program.
- Script mode: save code in a .py file and execute it with python3 filename.py — this is how every tool you build in this lab will run.

Python scripts intended to be run directly on Linux (as most security tools are) conventionally begin with a shebang line, a concept you already met in Lab 02:

```
#!/usr/bin/env python3
"""A short docstring describing what this script does."""

print("Hello, Cyber Security!")
```

A minimal Python script. Make it executable with chmod +x script.py, then run it directly with ./script.py.

B.2 Variables and Dynamic Typing

In C++, every variable is declared with an explicit type that cannot change: int x = 5; permanently makes x an integer. Python variables are names bound to objects, and that binding can be rebound to a different type at any time. This is called dynamic typing.

```
x = 5              # x is bound to an int
print(type(x))     # <class 'int'>

x = "five"         # the SAME name x is now bound to a str
print(type(x))     # <class 'str'>

# This would be a compile error in C++ (cannot reassign int to string)
# but is perfectly legal, if occasionally dangerous, in Python.
```

Why this matters for tool-writing

Dynamic typing speeds up prototyping but removes a safety net C++ gives you for free. Security tools that parse untrusted input (a banner string, an HTTP response, a log line) must validate types and values explicitly in Python, because the interpreter will not stop you from, say, trying to add an integer to a string you assumed was a number.

B.3 Core Data Types

Python Type	Example	Closest C++ Equivalent
`int`	port = 443	int (Python ints have arbitrary precision — no overflow)
`float`	timeout = 1.5	`double`

Python Type	Example	Closest C++ Equivalent
`str`	host = "scanme.nmap.org"	std::string
`bool`	is_open = True	`bool`
NoneType	result = None	nullptr (conceptually)

B.4 Collections: list, tuple, dict, set

Python's four built-in collection types replace most of the containers you used from the C++ Standard Template Library. The mapping is direct enough to reason about immediately:

Python	C++ (STL) Equivalent	Mutable?
list — ports = [21, 22, 80]	std::vector<int>	Yes
tuple — addr = ("10.0.0.5", 80)	std::array / a returned struct	No (immutable)
dict — services = {80: "http"}	std::map / std::unordered_map	Yes
set — open_ports = {22, 80, 443}	std::set / std::unordered_set	Yes

```
# list: ordered, mutable, like std::vector
common_ports = [21, 22, 23, 25, 80, 443]
common_ports.append(8080)
print(common_ports[0])          # 21  (zero-indexed, same as C++)

# tuple: ordered, IMMUTABLE -- perfect for a fixed (host, port) pair
target = ("scanme.nmap.org", 80)
host, port = target             # "unpacking" -- no equivalent in raw C++

# dict: key -> value, like std::map but with dynamic keys/values
port_services = {21: "FTP", 22: "SSH", 80: "HTTP", 443: "HTTPS"}
print(port_services[22])        # "SSH"
port_services[8080] = "HTTP-Proxy"

# set: unique, unordered values -- useful for de-duplicating scan results
open_ports = {22, 80, 443}
open_ports.add(80)              # no effect, 80 is already present
print(len(open_ports))          # 3
```

The four collection types you will use throughout the rest of this lab.

B.5 String Formatting

Python's f-strings (formatted string literals) are the modern equivalent of using std::ostringstream or sprintf to build a formatted string — but far more concise:

```
host = "192.168.56.101"
port = 22
status = "open"

# f-string: embed expressions directly inside {}
print(f"[{status.upper()}] {host}:{port}")
# Output: [OPEN] 192.168.56.101:22

# Equivalent, more verbose C++:
# std::cout << "[" << status << "] " << host << ":" << port << std::endl;
```

B.6 Input and Output

print() is the direct equivalent of std::cout, and input() is the direct equivalent of std::cin — with the useful difference that input() always returns a str, which you must explicitly convert (cast) if you need a number:

```
target = input("Enter target IP: ")       # always returns a string
port = int(input("Enter port number: "))  # explicit conversion, like C++'s std::stoi()

print("Scanning", target, "on port", port, sep=" ", end="\n")
```

Part C — Control Flow, Functions, and Exception Handling

C.1 Conditionals

if / elif / else works exactly as if / else if / else does in C++, with two syntax changes: no parentheses are required around the condition, and the block is defined by indentation rather than braces. One genuine trap for C++ programmers: Python has both == (value equality, what you want almost always) and is (identity — "is this the exact same object in memory", rarely what you want for comparing values).

```
port = 443

if port == 80:
    print("Standard HTTP")
elif port == 443:
    print("Standard HTTPS")
else:
    print("Non-standard port")

# Common bug for C++ programmers: use == to compare values, not 'is'
a = 1000
b = 1000
print(a == b)   # True  -- correct way to compare values
# print(a is b) # may print False -- 'is' checks object identity, not value
```

C.2 Loops

Python's for loop iterates directly over a sequence — there is no C-style for (int i = 0; i < n; i++) counting loop by default. To reproduce that pattern, use range():

```
# C++: for (int port = 1; port <= 5; port++) { ... }
for port in range(1, 6):        # 1,2,3,4,5  (stop value is EXCLUSIVE)
    print(port)

# Iterating directly over a collection -- very common in security scripts
for port in [21, 22, 80, 443]:
    print(f"Checking port {port}")

# while loop -- identical logic to C++
attempts = 0
while attempts < 3:
    print("Retrying connection...")
    attempts += 1               # Python has no ++ operator; use += 1

# break and continue work exactly as in C++
```

C.3 Functions

def replaces a C++ function signature. Python functions do not declare a return type, and can return multiple values at once by returning a tuple — a very common pattern that replaces C++'s use of output parameters (passing by reference) or a returned struct.

```
def scan_port(host, port, timeout=1.0):
    """Returns (port, is_open) -- a tuple, unpacked by the caller."""
    import socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(timeout)
    result = sock.connect_ex((host, port))   # 0 means the port is open
    sock.close()
    return port, (result == 0)

# Calling the function and unpacking its tuple return value
port_number, is_open = scan_port("127.0.0.1", 22)
print(f"Port {port_number} open: {is_open}")

# *args and **kwargs -- variable-length argument lists
# (no direct C++ equivalent short of variadic templates)
def scan_many(host, *ports, **options):
    verbose = options.get("verbose", False)
    for p in ports:
        if verbose:
            print(f"Scanning {host}:{p}")

scan_many("127.0.0.1", 22, 80, 443, verbose=True)
```

Default arguments (timeout=1.0 above) replace C++'s default parameter values and work identically. Keyword arguments (verbose=True) let a caller name a parameter explicitly, improving readability for functions with many options — extremely common in security tools that expose many configurable flags.

C.4 Lambda Functions

A lambda is a small, anonymous, single-expression function — similar in spirit to a C++ lambda ([](int x){ return x*2; }), but usable inline anywhere a function object is expected, most often as a sort key:

```
scan_results = [(21, False), (22, True), (80, True), (443, False)]

# Sort so that open ports (True) come first
scan_results.sort(key=lambda result: result[1], reverse=True)
print(scan_results)   # [(22, True), (80, True), (21, False), (443, False)]
```

C.5 Exception Handling

Python's try / except / else / finally is the direct equivalent of C++'s try / catch, with one important addition: finally, a block that always runs whether or not an exception occurred — the natural place to close a socket or a file. C++ has no finally keyword; C++ programmers rely on RAII and destructors to achieve the same

guarantee. In Python, you must close resources explicitly (or use a with block, Python's equivalent of RAII, introduced when you build the port scanner).

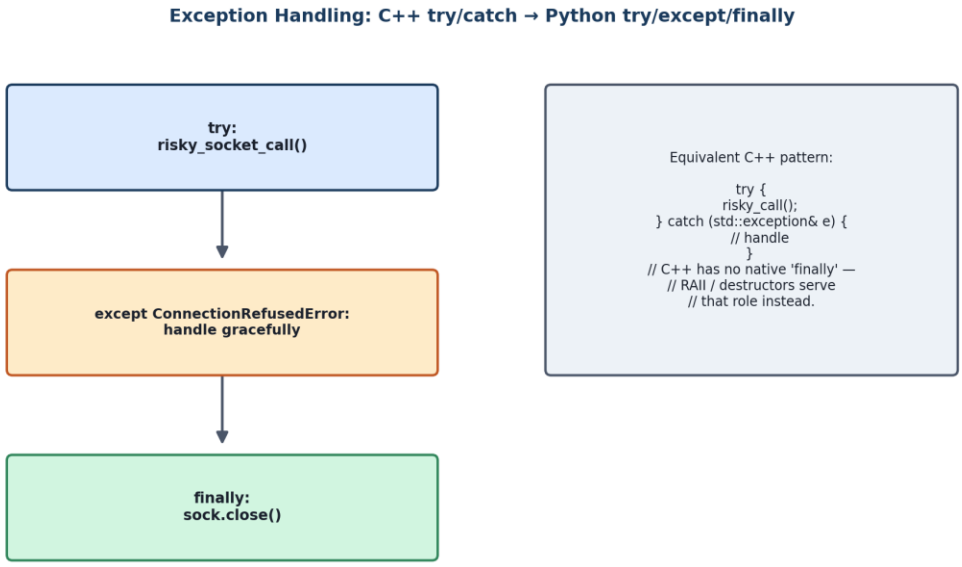


Figure 2: Python's try/except/finally mapped against the C++ try/catch pattern you already know.

```
import socket

def connect_to(host, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(2)
    try:
        sock.connect((host, port))
        print(f"Connected to {host}:{port}")
    except socket.timeout:
        print(f"[!] Connection to {host}:{port} timed out")
    except ConnectionRefusedError:
        print(f"[!] {host}:{port} refused the connection (port likely closed)")
    except socket.gaierror:
        print(f"[!] Could not resolve hostname: {host}")
    else:
        print("No exceptions -- connection succeeded cleanly")
    finally:
        sock.close()   # ALWAYS runs -- guarantees the socket is released

connect_to("scanme.nmap.org", 80)
```

Catching the specific exceptions a network connection can raise. Every socket program you write from this point on should follow this pattern.

Catch specific exceptions

Just as catching (...) in C++ hides the real error type, a bare except: in Python silently swallows every error — including ones you did not anticipate, such as a typo in your own code. Always catch the most specific exception class you expect

(socket.timeout, ConnectionRefusedError), and let genuinely unexpected exceptions propagate so you notice them during development.

Part D — Object-Oriented Programming in Python

You have already written classes, constructors, inheritance hierarchies, and polymorphic functions in C++. Python OOP is built on exactly the same ideas, expressed with different keywords and a more permissive attitude toward access control.

D.1 Classes, Objects, and Constructors

class replaces class. __init__ replaces the constructor. self is an explicit first parameter on every instance method, playing the same role as the implicit this pointer in C++ — the difference is that Python makes it visible in every method signature rather than hiding it.

```
class ScanTarget:
    """Represents a single host to be scanned -- like a small C++ class."""

    def __init__(self, host, ports):
        self.host = host          # equivalent to a member variable
        self.ports = ports
        self.open_ports = []      # populated later by scan()

    def scan(self):
        import socket
        for port in self.ports:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(0.5)
            if sock.connect_ex((self.host, port)) == 0:
                self.open_ports.append(port)
            sock.close()
        return self.open_ports

target = ScanTarget("127.0.0.1", [22, 80, 443])   # like ScanTarget target("127.0.0.1", ...);
print(target.scan())
```

D.2 Encapsulation

C++ enforces private / protected / public at compile time. Python has no compiler-enforced access control; instead it relies on convention and a small amount of name mangling:

Convention	Meaning	C++ Equivalent
`self.name`	Public — freely accessible	`public`
`self._name`	"Protected by convention" — accessible, but a signal to other developers: treat as internal	protected (weakly enforced)
`self.__name`	Name-mangled to _ClassName__name — discourages, but does not prevent, external access	private (weakly enforced)

D.3 Inheritance and Polymorphism

Python inheritance uses class Child(Parent): instead of class Child : public Parent {}. super().__init__(...) calls the parent constructor, replacing C++'s Parent::Parent(...) initializer-list call. Because Python is dynamically typed, every method is effectively virtual by default — there is no virtual keyword to remember, and no vtable to reason about explicitly.

```
class Scanner:
    def __init__(self, host):
        self.host = host

    def run(self):
        raise NotImplementedError("Subclasses must implement run()")


class TcpPortScanner(Scanner):          # inherits from Scanner
    def __init__(self, host, ports):
        super().__init__(host)          # calls Scanner.__init__()
        self.ports = ports

    def run(self):                      # overrides Scanner.run() -- no 'override' keyword
needed
        print(f"Running TCP scan against {self.host} on {len(self.ports)} ports")


scanner = TcpPortScanner("127.0.0.1", [22, 80, 443])
scanner.run()   # polymorphism: calls TcpPortScanner's run(), exactly as C++'s virtual
dispatch would
```

D.4 Useful Dunder ('Magic') Methods

Methods surrounded by double underscores let a class define how it behaves with built-in operators and functions — conceptually similar to operator overloading in C++ (operator<<, operator==).

Dunder Method	Triggered By	C++ Analogue
__init__(self, ...)	Object creation	Constructor
__str__(self)	print(obj) / str(obj)	operator<< (friend function)
__repr__(self)	Debug/console representation	A debug-print helper
__len__(self)	len(obj)	`.size()` member function
__eq__(self, other)	obj1 == obj2	operator==

Part E — Modules, Packages, and the Python Security Ecosystem

E.1 The Import System

Python's import replaces C++'s #include, but works at a higher level: instead of textually pasting header contents, import loads an actual module object at runtime. This is closer to linking against a compiled library than to a C++ #include.

```
import socket                    # import the whole module; call as socket.socket(...)
import socket as sk               # import with an alias
from socket import socket, AF_INET # import specific names directly into your namespace
from concurrent.futures import ThreadPoolExecutor as Pool
```

E.2 pip and Virtual Environments (Recap)

pip is Python's package manager — the equivalent of apt for system packages, but scoped to Python libraries. You already used it in Section 3.1 to install requests and cryptography inside a venv. Every third-party package referenced in this lab is installed the same way: pip install <package-name>.

E.3 Standard Library Highlights for Security Work

These modules ship with every Python installation — no pip install required. You will use most of them before the end of this lab.

Module	Purpose	Used Later In This Lab For
`os`	Operating-system interfaces: files, paths, environment variables	Walking directories for the file-integrity checker
`sys`	Interpreter internals: command-line arguments, exit codes	Reading argv, exiting with status codes
`subprocess`	Launch and control external programs (like Nmap itself)	Wrapping CLI tools from inside Python
`re`	Regular expressions — pattern matching in text	Parsing HTTP titles, log files
`hashlib`	Cryptographic hash functions (MD5, SHA-1, SHA-256, ...)	The file-integrity checker
`json`	Read/write JSON data	Saving scan results and integrity baselines
`argparse`	Parse command-line arguments into a clean interface	Every tool built in Parts H–K
`socket`	Low-level networking — TCP/UDP sockets	The port scanner and banner grabber
`concurrent.futures`	Thread and process pools for parallel work	Multithreading the port scanner
`collections`	Specialized containers (Counter, defaultdict, ...)	Counting failed logins in the log analyzer

E.4 Third-Party Security-Relevant Packages

These require pip install. Each is a cornerstone of some part of the security tooling ecosystem — you will use requests directly in this lab, and should recognize the others by name and purpose.

Package	install name	Role in Cyber Security
Requests	`requests`	The standard HTTP client library — used for web recon, API interaction, and OSINT scripts
Scapy	`scapy`	Craft, sniff, and manipulate raw network packets at any protocol layer
python-nmap	python-nmap	A Python wrapper around the Nmap binary — run scans and parse XML output programmatically
Paramiko	`paramiko`	Pure-Python SSH client/server — automate remote administration and SSH-based exploitation
`cryptography`	`cryptography`	Modern, high-level encryption, hashing, and certificate handling
BeautifulSoup	beautifulsoup4	Parse and extract data from HTML — used heavily in web scraping and OSINT

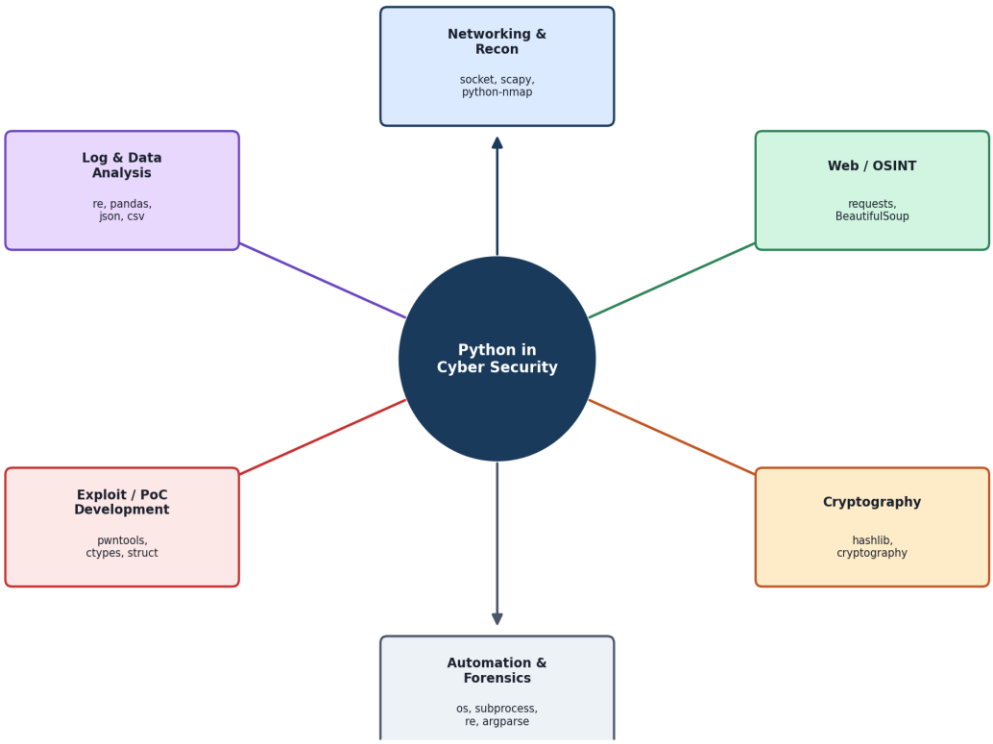


Figure 3: Python's role across the major functional areas of cyber security work, grouped by the modules typically used in each.

Part F — Networking with Python: Socket Programming Fundamentals

F.1 What Is a Socket?

A socket is an operating-system-provided endpoint for network communication — the same abstraction Nmap, your web browser, and every SSH client are ultimately built on. A socket is identified by an IP address and a port number, and Python's built-in socket module exposes this OS-level API almost directly, which is exactly why it maps so cleanly onto the theory you likely covered in networking coursework.

F.2 The TCP Three-Way Handshake

Every TCP connection — including the ones your port scanner will make — begins with the same three-packet exchange. Understanding it is the difference between using socket.connect() as a black box and understanding exactly what a port scanner is measuring.

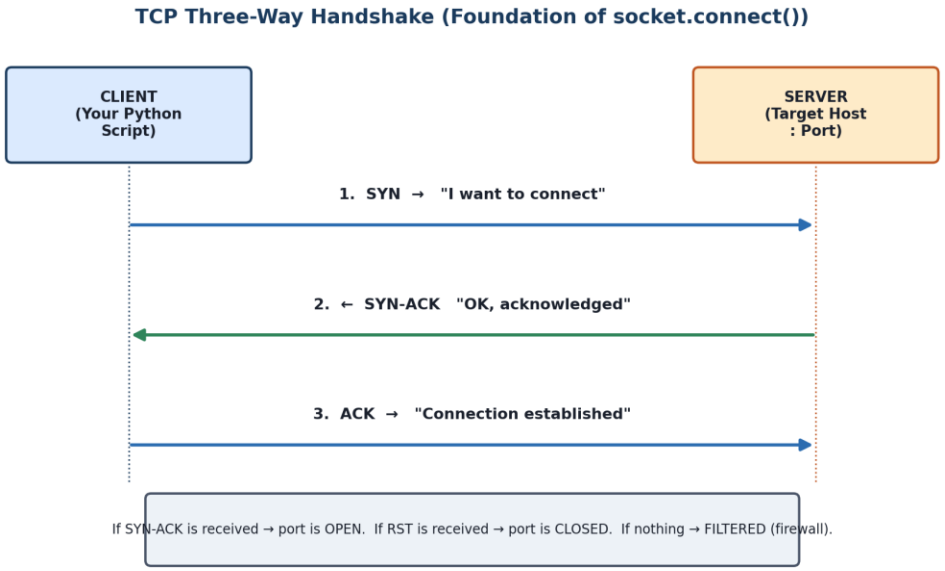


Figure 4: The TCP three-way handshake: SYN, SYN-ACK, ACK. A completed handshake is what socket.connect() waits for, and what tells a port scanner a port is open.

This is precisely the signal a port scanner is looking for: if the handshake completes, the port is open and a service is listening. If the target actively refuses the connection (sends a RST packet), the port is closed. If nothing comes back at all within a timeout window, the port is most likely filtered by a firewall.

F.3 The socket Module API

Function / Method	Purpose
socket.socket(family, type)	Create a new socket object. AF_INET = IPv4, SOCK_STREAM = TCP
sock.connect((host, port))	Connect to a remote host/port; raises an exception on failure
sock.connect_ex((host, port))	Same as connect(), but RETURNS an error code instead of raising — ideal for scanning many ports without a try/except around every attempt

Function / Method	Purpose
sock.settimeout(seconds)	Set how long to wait before giving up on a blocking call
sock.send(data) / sendall(data)	Send bytes over an established connection
sock.recv(bufsize)	Receive up to bufsize bytes from the connection
sock.bind((host, port))	(Server-side) Attach the socket to a local address/port
sock.listen(backlog)	(Server-side) Start listening for incoming connections
`sock.accept()`	(Server-side) Block until a client connects; returns a new socket for that client
`sock.close()`	Release the socket — always call this, ideally in a finally block or a with statement

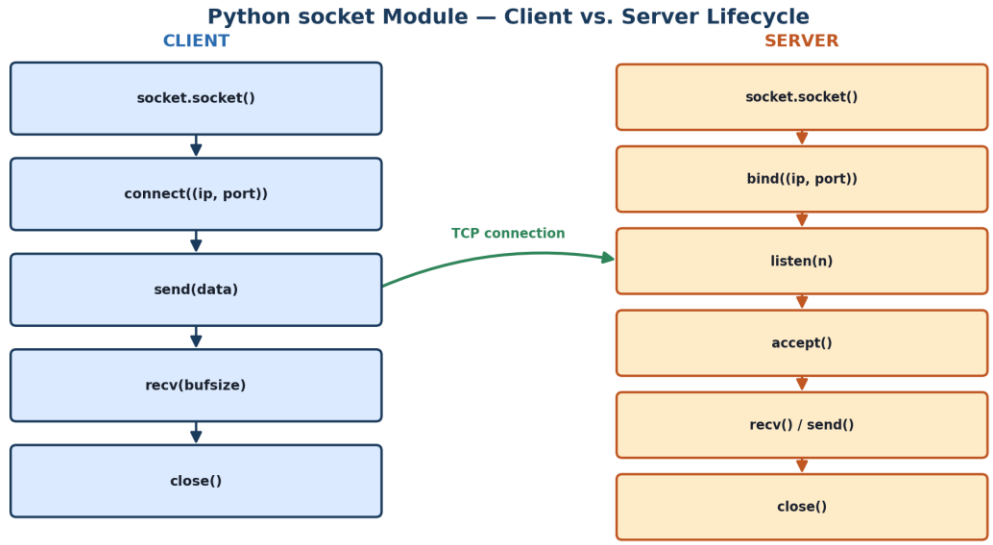


Figure 5: The client and server call sequences using Python's socket module. Notice the server has two extra setup steps (bind, listen) that the client does not need.

F.4 Writing a Simple TCP Client

This client connects to a web server on port 80, sends a minimal raw HTTP GET request, and prints the response — demonstrating that HTTP itself is just text sent over a TCP socket, the same socket primitive you are about to use to build a scanner.

```
#!/usr/bin/env python3
import socket
```

```
def fetch_raw_http(host, port=80, path="/"):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(5)
        sock.connect((host, port))
        request = f"GET {path} HTTP/1.1\r\nHost: {host}\r\nConnection: close\r\n\r\n"
        sock.sendall(request.encode())

        response = b""
        while True:
            chunk = sock.recv(4096)
            if not chunk:
                break
            response += chunk
        return response.decode(errors="replace")

if __name__ == "__main__":
    print(fetch_raw_http("scanme.nmap.org"))
```

with socket.socket(...) as sock: is Python's context manager syntax — the closest equivalent to C++ RAII. It guarantees sock.close() is called automatically, even if an exception occurs.

F.5 Writing a Simple TCP Echo Server

Run this in one terminal, then connect to it from a second terminal using nc localhost 9999 or a small client script — a useful safe target for testing your own tools later in this lab.

```
#!/usr/bin/env python3
import socket

HOST, PORT = "127.0.0.1", 9999

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    print(f"[*] Echo server listening on {HOST}:{PORT}")

    while True:
        conn, addr = server.accept()
        with conn:
            print(f"[+] Connection from {addr}")
            data = conn.recv(1024)
            if data:
                conn.sendall(b"ECHO: " + data)
```

A minimal TCP server: bind -> listen -> accept -> recv/send -> close, matching the server column of Figure 5.

Hands-on checkpoint

Run the echo server above in one terminal. In a second terminal, run: python3 -c "import socket; s=socket.socket();

s.connect(('127.0.0.1',9999)); s.send(b'hello'); print(s.recv(1024))" and confirm you see the echoed response. This confirms your socket fundamentals are working before you move on to scanning.

Part G — Introducing Nmap: The Industry-Standard Port Scanner

G.1 What Is Nmap, and Why Learn It First?

Nmap ("Network Mapper") is the tool nearly every course, certification, and professional workflow uses as the reference implementation of network scanning. Before you write your own scanner in Part H, you should understand what a mature, production-grade tool does — both so you have a correctness baseline to compare your own tool's output against, and so you understand precisely what problem your own code will be solving. Nmap ships pre-installed on Kali Linux.

```
nmap --version
```

G.2 Legal & Ethical Reminder

Only scan what you are authorized to scan

For this lab, restrict every Nmap scan to: 127.0.0.1 (your own machine), your lab VM's private IP (e.g. inside a VirtualBox Host-Only or NAT network), or scanme.nmap.org — a host the Nmap project explicitly makes available for learners to practice against. Do not scan any other public IP address or domain.

G.3 Core Nmap Syntax

Flag	Meaning
-sS	SYN scan ("stealth"/half-open scan) — sends SYN, reads the response, never completes the handshake
-sT	Full TCP connect scan — completes the full three-way handshake, exactly like our Python client
-sU	UDP scan
-sV	Probe open ports to determine the service/version running on them
-O	Attempt operating-system fingerprinting
-A	Aggressive scan: enables -sV, -O, script scanning, and traceroute together
-p <ports>	Specify ports/ranges, e.g. -p 22,80,443 or -p 1-1000
-Pn	Skip host-discovery (ping) and scan even if the host appears down
--top-ports N	Scan only the N most common ports
-oN file.txt	Save normal-format output to a file

G.4 Hands-On: Your First Authorized Scan

```
nmap -sV -p 1-1000 scanme.nmap.org
```

Sample (abbreviated) output you should expect to see something similar to:

```
Starting Nmap 7.9x ( https://nmap.org ) at 2026-08-23 10:00 PKT
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.21s latency).
Not shown: 995 filtered ports
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 6.6.1p1
80/tcp   open  http    Apache httpd 2.4.7
9929/tcp open  nping-echo Nping echo
```

Read this output the same way you will read your own scanner's output later: PORT/protocol, STATE (open, closed, or filtered), SERVICE (Nmap's guess at what protocol runs there), and optionally VERSION when -sV is used. Record your own results in your lab notebook as part of this task.

G.5 The Bridge to Part H

Here is the key insight this whole lab has been building toward: what Nmap's -sT scan does internally is exactly the socket.connect_ex() loop you wrote in Part F, repeated across a range of ports, made fast through concurrency, and wrapped in a polished command-line interface. You now have every individual piece needed to build a real, working version of that idea yourself.

Part H — Building Your Own Tool: A Multithreaded Python Port Scanner

H.1 Design Overview

Before writing code, define the requirements, exactly as you would before starting a C++ project:

- Accept a target host and a port range from the command line (not hard-coded).
- Attempt a TCP connection to each port and record whether it is open.
- Run the checks concurrently, so scanning 1,000 ports does not take 1,000 sequential timeouts.
- Print a clean, readable report of every open port found.

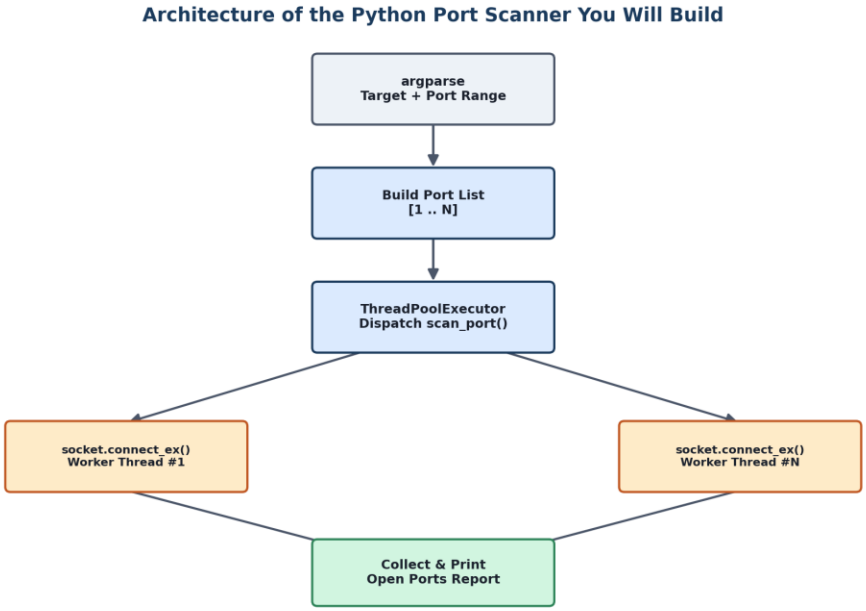


Figure 6: The architecture you will implement: parse arguments, build a port list, dispatch each port to a worker thread, and collect results.

H.2 Step 1 — A Single-Port Checker Function

This is the atomic unit of work — identical in spirit to the scan_port() function you already saw in Part C, refined to use connect_ex() so that a closed port does not require a try/except:

```
import socket

def scan_port(host, port, timeout=1.0):
    """Return True if the TCP port is open, False otherwise."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(timeout)
        result = sock.connect_ex((host, port))   # 0 == success == port open
        return result == 0
```

H.3 Step 2 — A Command-Line Interface with argparse

argparse turns your script into a real command-line tool, the same way a well-designed C++ program parses argc/argv — except argparse also generates --help output, validates types, and reports usage errors automatically.

```
import argparse

def build_parser():
    parser = argparse.ArgumentParser(description="A simple multithreaded TCP port scanner.")
    parser.add_argument("target", help="Target hostname or IP address")
    parser.add_argument("-p", "--ports", default="1-1024",
                        help="Port range, e.g. 1-1024 or a comma list 22,80,443")
    parser.add_argument("-t", "--threads", type=int, default=100,
                        help="Number of concurrent worker threads (default: 100)")
    parser.add_argument("--timeout", type=float, default=1.0,
                        help="Per-port connection timeout in seconds (default: 1.0)")
    return parser

def parse_ports(port_arg):
    """Turn '1-1024' or '22,80,443' into a list of integers."""
    if "-" in port_arg:
        start, end = port_arg.split("-")
        return list(range(int(start), int(end) + 1))
    return [int(p) for p in port_arg.split(",")]
```

H.4 Step 3 — Multithreading with concurrent.futures

Python threads are well-suited to this task because scanning is I/O-bound — each worker spends almost all its time waiting on the network, not using the CPU, so Python's Global Interpreter Lock (a detail beyond this lab's scope) is not the bottleneck it would be for CPU-bound work. ThreadPoolExecutor is the standard-library tool for exactly this pattern:

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def run_scan(target, ports, threads, timeout):
    open_ports = []
    with ThreadPoolExecutor(max_workers=threads) as executor:
        future_to_port = {
            executor.submit(scan_port, target, port, timeout): port
            for port in ports
        }
        for future in as_completed(future_to_port):
            port = future_to_port[future]
            if future.result():
                open_ports.append(port)
                print(f"[+] {target}:{port} is OPEN")
    return sorted(open_ports)
```

H.5 Full Program

Assembled into one file, port_scanner.py:

```
#!/usr/bin/env python3
"""
port_scanner.py -- A simple multithreaded TCP port scanner.
Educational use only. Scan only hosts you are authorized to scan.
"""
import socket
import argparse
import sys
import time
from concurrent.futures import ThreadPoolExecutor, as_completed


def scan_port(host, port, timeout=1.0):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(timeout)
        return sock.connect_ex((host, port)) == 0


def parse_ports(port_arg):
    if "-" in port_arg:
        start, end = port_arg.split("-")
        return list(range(int(start), int(end) + 1))
    return [int(p) for p in port_arg.split(",")]


def run_scan(target, ports, threads, timeout):
    open_ports = []
    with ThreadPoolExecutor(max_workers=threads) as executor:
        future_to_port = {executor.submit(scan_port, target, p, timeout): p for p in ports}
        for future in as_completed(future_to_port):
            port = future_to_port[future]
            if future.result():
                open_ports.append(port)
                print(f"[+] {target}:{port} is OPEN")
    return sorted(open_ports)


def main():
    parser = argparse.ArgumentParser(description="A simple multithreaded TCP port scanner.")
    parser.add_argument("target", help="Target hostname or IP address")
    parser.add_argument("-p", "--ports", default="1-1024", help="e.g. 1-1024 or 22,80,443")
    parser.add_argument("-t", "--threads", type=int, default=100)
    parser.add_argument("--timeout", type=float, default=1.0)
    args = parser.parse_args()

    try:
```

```
        target_ip = socket.gethostbyname(args.target)
    except socket.gaierror:
        print(f"[!] Could not resolve host: {args.target}")
        sys.exit(1)

    ports = parse_ports(args.ports)
    print(f"[*] Scanning {args.target} ({target_ip}) -- {len(ports)} ports, {args.threads}
threads")

    start = time.time()
    open_ports = run_scan(target_ip, ports, args.threads, args.timeout)
    elapsed = time.time() - start

    print(f"\n[*] Scan complete in {elapsed:.2f}s -- {len(open_ports)} open port(s):
{open_ports}")


if __name__ == "__main__":
    main()
```

Save as port_scanner.py, run: chmod +x port_scanner.py && ./port_scanner.py scanme.nmap.org -p 1-1000 -t 200

H.6 Sample Run

```
$ ./port_scanner.py scanme.nmap.org -p 1-1000 -t 200
[*] Scanning scanme.nmap.org (45.33.32.156) -- 1000 ports, 200 threads
[+] scanme.nmap.org:22 is OPEN
[+] scanme.nmap.org:80 is OPEN

[*] Scan complete in 4.31s -- 2 open port(s): [22, 80]
```

Compare this against your Nmap results from Part G.4 — the open ports reported should match. Any discrepancy is worth investigating: it usually means a firewall is behaving differently toward Nmap's raw-socket SYN scan than toward your script's full-connect scan, or that your timeout is too aggressive for a slow link.

H.7 Suggested Enhancements (Try These)

- Add a --common-ports flag that scans only a curated list of the 20 most-attacked ports instead of a full range.
- Write results to a file with --output results.json using the json module.
- Add a progress indicator using tqdm or a simple counter, so long scans give feedback.
- Hook in the banner grabber you will build in Part I so each open port also reports its service banner in one pass.

Part I — Banner Grabbing and the Title-Grabber Tool

I.1 What Is Banner Grabbing, and Why Does It Matter?

Knowing a port is open is only half the picture. Many services announce themselves the moment a connection is made — an SSH server sends its version string immediately, an FTP server sends a welcome banner, and an SMTP server sends its greeting. Capturing this text, called banner grabbing, is one of the most important reconnaissance steps in a penetration test, because a specific software version (e.g. "OpenSSH 6.6.1p1") can be checked directly against known CVEs. This is precisely what Nmap's -sV flag automates.

I.2 A Generic TCP Banner Grabber

This works for any protocol that sends data immediately after the handshake (SSH, FTP, SMTP, many others):

```
#!/usr/bin/env python3
import socket

def grab_banner(host, port, timeout=2.0):
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.settimeout(timeout)
            sock.connect((host, port))
            banner = sock.recv(1024).decode(errors="replace").strip()
            return banner if banner else "(no banner received)"
    except (socket.timeout, ConnectionRefusedError, OSError) as e:
        return f"(error: {e})"

if __name__ == "__main__":
    for port in (21, 22, 25, 80):
        print(f"{port}: {grab_banner('scanme.nmap.org', port)}")
```

Note this reuses the exact exception-handling pattern from Part C.5 — network code always needs it.

I.3 An HTTP Title Grabber

HTTP does not send a banner on connect — you must send a request first, exactly as in Part F.4. The <title> tag of the returned HTML is a common, lightweight fingerprint ("Apache2 Ubuntu Default Page", "Welcome to nginx", a CMS login page title, and so on). This version uses the requests library, which handles connection details for you, plus a regular expression to extract the title:

```
#!/usr/bin/env python3
import re
import requests

def grab_title(url, timeout=5):
    try:
        response = requests.get(url, timeout=timeout)
```

```
        match = re.search(r"<title>(.*?)</title>", response.text, re.IGNORECASE | re.DOTALL)
        title = match.group(1).strip() if match else "(no <title> tag found)"
        return response.status_code, title
    except requests.exceptions.RequestException as e:
        return None, f"(request failed: {e})"

if __name__ == "__main__":
    for url in ("http://scanme.nmap.org", "http://127.0.0.1"):
        status, title = grab_title(url)
        print(f"{url} -> [{status}] {title}")
```

pip install requests first if you have not already. re.DOTALL lets the '.' in the pattern match across newlines, since HTML titles occasionally span lines.

Guided exercise: build your own title grabber

Extend the script above so that it: (1) reads a list of URLs from a text file passed as a command-line argument via argparse, (2) records the HTTP status code, page title, and response time for each, (3) prints a clean aligned report, and (4) gracefully reports any URL that fails to connect rather than crashing the whole run. This exact task also appears as Graded Exercise 3 in Part K — build it here first as practice.

Part J — Other Cyber Security Use Cases for Python

Port scanning and banner grabbing are only two applications among many. This section is a guided tour of five more roles Python plays across the industry, each with a working, runnable example.

J.1 File Integrity Verification with hashlib

A cryptographic hash acts as a fixed-length fingerprint of a file's exact contents — the basis of file-integrity monitoring (FIM), malware signature checks, and verifying a downloaded ISO was not tampered with.

```
import hashlib

def sha256_of_file(path, block_size=65536):
    sha256 = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(block_size), b""):
            sha256.update(block)
    return sha256.hexdigest()

print(sha256_of_file("/etc/hostname"))
```

J.2 Automating External Tools with subprocess

Rather than reimplementing Nmap, professional Python tools often wrap it — launching it as a subprocess and parsing its output, which is exactly how the python-nmap package from Part E.4 works internally.

```
import subprocess

result = subprocess.run(
    ["nmap", "-sV", "-p", "22,80", "127.0.0.1"],
    capture_output=True, text=True, timeout=30
)
print(result.stdout)
```

J.3 Log Parsing and Analysis with re

Security analysts spend a large fraction of their time turning raw log text into structured, countable data. Regular expressions are the standard tool for that extraction, and Python's collections.Counter makes tallying results trivial:

```
import re
from collections import Counter

log_lines = [
    "Aug 23 09:12:01 srv sshd[1022]: Failed password for root from 203.0.113.5 port 51422",
    "Aug 23 09:12:04 srv sshd[1023]: Failed password for admin from 203.0.113.5 port 51500",
    "Aug 23 09:13:10 srv sshd[1024]: Failed password for root from 198.51.100.9 port 40011",
    "Aug 23 09:14:00 srv sshd[1025]: Accepted password for annus from 192.168.56.1 port
```

```
22334",
]

pattern = re.compile(r"Failed password for \S+ from (\d+\.\d+\.\d+\.\d+)")
failed_ips = Counter(m.group(1) for line in log_lines if (m := pattern.search(line)))

for ip, count in failed_ips.most_common():
    print(f"{ip}: {count} failed attempt(s)")
```

This is a miniature version of Graded Exercise 4 in Part K.

J.4 Basic Web Reconnaissance with requests

Checking response headers is a fast, non-intrusive way to learn about a web server's configuration and whether it is missing common security headers:

```
import requests

response = requests.get("http://scanme.nmap.org", timeout=5)
print("Server header:", response.headers.get("Server", "(not disclosed)"))

security_headers = ["X-Frame-Options", "Content-Security-Policy", "Strict-Transport-
Security"]
for header in security_headers:
    present = header in response.headers
    print(f"{header}: {'present' if present else 'MISSING'}")
```

J.5 Packet Inspection with Scapy (Conceptual Overview)

Scapy operates one layer below socket — it lets you construct and inspect individual packets field-by-field, which is why it is the standard library for protocol research, custom packet crafting, and traffic analysis. It typically requires root privileges because it works with raw sockets. A safe, read-only starting point is inspecting a packet capture file you already own rather than sending live traffic:

```
from scapy.all import rdpcap

packets = rdpcap("sample_capture.pcap")   # a .pcap file you already have permission to
analyze
print(f"{len(packets)} packets in capture")
for pkt in packets[:5]:
    print(pkt.summary())
```

Building and sending custom packets (ARP requests, crafted TCP flags, and so on) is a natural next step once you are comfortable with the fundamentals in this lab, but always requires an isolated lab network and explicit authorization — crafted traffic can disrupt real infrastructure if aimed anywhere else.

J.6 Symmetric Encryption with cryptography

The cryptography package's Fernet class provides authenticated symmetric encryption with a sane, hard-to-misuse default configuration — useful for protecting configuration secrets, exfiltration-test payloads in an authorized red-team exercise, or simply understanding how encrypted C2 channels work at a basic level.

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()          # keep this secret -- it is both the lock and the key
cipher = Fernet(key)

token = cipher.encrypt(b"top secret scan results")
print("Encrypted:", token)

plaintext = cipher.decrypt(token)
print("Decrypted:", plaintext.decode())
```

J.7 A Note on Higher-Risk Tool Categories

You will encounter references, in security literature and in real engagements, to tool categories such as keyloggers, remote access trojans (RATs), and credential-harvesting scripts. These exist for legitimate purposes — authorized red-team engagements, incident-response research, and detection-signature development — but they are also the same building blocks used in real attacks. This lab intentionally does not provide working implementations of these categories. If your future coursework, capstone project, or a Capture-The-Flag competition requires you to build one, that work must happen only inside an isolated lab environment, under explicit written authorization, and ideally under the direct supervision of a faculty member or engagement lead.

Part K — Hands-On Graded Exercises

Complete all five exercises below. Submit each as a separate, well-commented .py file named exN_description.py (e.g. ex2_port_scanner.py), plus a short README.md summarizing how to run each script and any sample output you captured. Refer to the Deliverables & Rubric table at the end of this manual for mark distribution.

Exercise 1: C++-to-Python Warm-Up: Translating a Cipher

Objective: Demonstrate fluent translation of control flow and functions from C++ to Python by porting a simple XOR cipher.

Requirements:

- You are given (write it yourself first, in C++, using what you already know) a function that XOR-encrypts a std::string against a single-byte key, character by character.
- Reimplement the identical logic in Python as a function xor_cipher(data: str, key: int) -> str.
- The function must be reversible: calling it twice with the same key returns the original string (a property of XOR).
- Include a __main__ block that encrypts a sample string, prints the (unreadable) result, then decrypts it back and prints the original.

Deliverable: ex1_xor_cipher.py, plus the original C++ version you translated from, submitted side by side for comparison.

Hints: *Use ord() and chr() to move between characters and their integer values — the Python equivalents of C++'s implicit char/int conversion.*

Exercise 2: Extend the Port Scanner: Banner Grabbing + CSV Output

Objective: Combine Part H's scanner with Part I's banner grabber, and add structured output.

Requirements:

- Start from the port_scanner.py built in Part H.5.
- For every port found open, immediately attempt a banner grab (reuse grab_banner() from Part I.2) and store the result alongside the port number.
- Add a --output results.csv argument; write one row per open port containing: target, port, banner, timestamp.
- Handle the case where a banner grab times out gracefully — the port should still be reported as open with a banner of "(none)".

Deliverable: ex2_port_scanner.py and one sample results.csv generated from a real scan of an authorized target.

Hints: *Use Python's csv module (import csv) rather than manually formatting comma-separated strings, to correctly handle any commas that appear inside a banner.*

Exercise 3: Bulk HTTP Title Grabber

Objective: Build a production-shaped version of the guided exercise from Part I.3.

Requirements:

- Accept a text file of URLs (one per line) via argparse, e.g. ./title_grabber.py urls.txt.
- For each URL, record: HTTP status code, page title (or an explanatory placeholder if none found), and response time in milliseconds.
- Use a ThreadPoolExecutor (as in Part H.4) so multiple URLs are fetched concurrently rather than one at a time.
- Print a neatly aligned report; do not let one failed URL stop the rest of the run.

Deliverable: ex3_title_grabber.py plus a urls.txt containing at least 5 test URLs (a mix of reachable and deliberately unreachable ones, to demonstrate your error handling).

Hints: *f-strings support column-width formatting directly: f'{title:<40}' left-pads/truncates a field to 40 characters for clean alignment.*

Exercise 4: SSH Auth Log Analyzer

Objective: Extract and rank failed SSH login attempts from raw log text, extending the pattern shown in Part J.3.

Requirements:

- Use the sample log excerpt provided below (copy it into a file named sample_auth.log).
- Extract every IP address associated with a “Failed password” line using a regular expression.
- Report the IPs sorted by descending number of failed attempts, using collections.Counter.
- Additionally report the single username most frequently targeted across all failed attempts.

Deliverable: ex4_log_analyzer.py, run against the sample log, with your program's printed output included in your README.

Hints: *collections.Counter.most_common() returns results already sorted by frequency — you do not need to sort manually.*

```
Aug 23 09:12:01 srv sshd[1022]: Failed password for root from 203.0.113.5 port 51422 ssh2
Aug 23 09:12:04 srv sshd[1023]: Failed password for admin from 203.0.113.5 port 51500 ssh2
Aug 23 09:12:09 srv sshd[1024]: Failed password for root from 203.0.113.5 port 51610 ssh2
Aug 23 09:13:10 srv sshd[1025]: Failed password for root from 198.51.100.9 port 40011 ssh2
Aug 23 09:13:44 srv sshd[1026]: Failed password for oracle from 198.51.100.9 port 40090 ssh2
Aug 23 09:14:00 srv sshd[1027]: Accepted password for annus from 192.168.56.1 port 22334 ssh2
Aug 23 09:15:12 srv sshd[1028]: Failed password for root from 203.0.113.5 port 51800 ssh2
Aug 23 09:16:40 srv sshd[1029]: Failed password for admin from 45.33.32.156 port 33221 ssh2
```

Sample auth-log excerpt for Exercise 4 (save as sample_auth.log).

Exercise 5: File Integrity Checker

Objective: Build a two-mode tool that establishes a hash baseline for a directory and later detects tampering.

Requirements:

- Mode 1 (--baseline <folder>): recursively walk every file in the given folder (use os.walk), compute the SHA-256 of each (reuse sha256_of_file() from Part J.1), and save {filepath: hash} as JSON to baseline.json.
- Mode 2 (--check <folder>): recompute hashes for the same folder and compare against baseline.json.
- Report three categories clearly: files that were MODIFIED (hash changed), files that are NEW (not in baseline), and files that are MISSING (in baseline but no longer present).
- Use argparse with mutually-exclusive --baseline / --check flags so the tool has one clear mode per run.

Deliverable: ex5_integrity_checker.py, a sample baseline.json, and a short demonstration where you modify one file and show the tool correctly flags it.

Hints: *argparse.add_mutually_exclusive_group() is the clean way to enforce exactly one of two flags being provided.*

Self-Evaluation

Skills Checklist

Before submitting this lab, honestly check off every item you can do without referring back to the manual:

- ☐ I can explain, in my own words, the difference between compiled and interpreted execution.
- ☐ I can write a Python function with default arguments and a tuple return value.
- ☐ I can write a try/except/finally block that catches specific network exceptions.
- ☐ I can define a Python class with inheritance and explain how self relates to C++'s this.
- ☐ I can explain the TCP three-way handshake and how it relates to socket.connect().
- ☐ I can write a basic TCP client and a basic TCP server using the socket module.
- ☐ I can run an authorized Nmap scan and correctly read its output.
- ☐ I can explain, line by line, how my own port scanner determines whether a port is open.
- ☐ I can write a regular expression to extract a specific pattern (an IP address, an HTML tag) from text.
- ☐ I can explain at least three additional cyber security use cases for Python beyond scanning.

Conceptual Questions

Answer each of the following in 2–4 sentences in your submitted README.

1. Why is Python's lack of a compilation step an advantage for security tool development, and what is the corresponding disadvantage?
2. Explain the difference between == and is in Python, and describe a scenario where confusing the two could produce a subtle bug in a security script.
3. What does connect_ex() return that makes it more suitable than connect() for writing a port scanner?
4. In the TCP three-way handshake, what specifically distinguishes an OPEN port's response from a CLOSED port's response, and from a FILTERED port's (non-)response?
5. Why does the port scanner in Part H use threads rather than running each port check sequentially, and why are threads (rather than, say, separate processes) an appropriate tool for this specific job?
6. What is the practical security value of banner grabbing beyond simply knowing a port is open?
7. Give one example (not from this manual) of a legitimate, authorized use for a Python packet-crafting or sniffing script.
8. Why does this manual decline to provide working keylogger or credential-harvesting code, even though such tools exist in real security work?

Practical Self-Test Tasks

Quick, ungraded tasks to confirm your understanding before attempting the graded exercises:

- Modify the port scanner to accept multiple targets (a comma-separated list) instead of just one.
- Modify the TCP echo server so it can handle more than one client without exiting after the first connection.

- Add a --json flag to any tool in this lab that switches its output from plain text to JSON.
- Write a five-line script using hashlib that prints the MD5 and SHA-256 of a string you type in, side by side.
- Time how long your port scanner takes to scan ports 1–1000 with 10 threads versus 200 threads, and explain the difference you observe.

Self-Grading Guide

Use this guide to sanity-check your own work before submission. If you cannot honestly answer “yes” to a row, revisit that section of the manual.

Check	Expected Result
Does ex1 correctly reverse its own encryption?	Running it twice with the same key reproduces the original string exactly
Does ex2's scanner match your Part G.4 Nmap results?	The same set of open ports is reported by both tools on the same target
Does ex3 handle an unreachable URL without crashing?	The program prints an error for that URL and continues to the next one
Does ex4's IP ranking match a manual count of the sample log?	203.0.113.5 should be reported with the highest failed-attempt count (4)
Does ex5 correctly detect a modified file?	After editing one file post-baseline, --check reports it under MODIFIED, not MISSING or unchanged

Deliverables & Rubric

Submit a single compressed archive (yourRollNumber_Lab03.zip) containing all five exercise scripts, any supporting sample files (urls.txt, sample_auth.log, baseline.json), and a README.md documenting how to run each tool and what output you observed.

Component	Weight	Criteria
Exercise 1 — XOR Cipher Translation	10%	Correct, reversible logic; idiomatic Python; clear C++ comparison
Exercise 2 — Scanner with Banner Grabbing + CSV	25%	Correct threading, accurate results matching Nmap, valid CSV output, graceful error handling
Exercise 3 — Bulk Title Grabber	20%	Concurrent execution, correct status/title/timing extraction, robust error handling
Exercise 4 — Log Analyzer	15%	Correct regex extraction, accurate ranking, correct most-targeted-username logic
Exercise 5 — File Integrity Checker	20%	Correct baseline/check modes, accurate MODIFIED/NEW/MISSING classification
Code Quality & README	10%	Comments, meaningful names, argparse usage, clear run instructions, honest self-evaluation answers

References

- Python Software Foundation. Python 3 Documentation — https://docs.python.org/3/
- Python Software Foundation. socket — Low-level networking interface — https://docs.python.org/3/library/socket.html
- Python Software Foundation. concurrent.futures — Launching parallel tasks — https://docs.python.org/3/library/concurrent.futures.html
- Lyon, G. Nmap Network Scanning: Official Nmap Project Guide, and the Nmap Reference Guide — https://nmap.org/book/
- Postel, J. RFC 793: Transmission Control Protocol — Internet Engineering Task Force.
- Requests: HTTP for Humans — https://requests.readthedocs.io/
- Python Cryptography Authors. cryptography package documentation — https://cryptography.io/
- The Scapy Project. Scapy Documentation — https://scapy.readthedocs.io/
- OWASP Foundation. OWASP Testing Guide — https://owasp.org/www-project-web-security-testing-guide/
- Nmap Project. scanme.nmap.org — Authorized public scanning target — https://nmap.org/